

07 · System Prompt 的构建、资源发现与动态覆盖

默认提示词、SYSTEM.md、APPEND_SYSTEM.md、AGENTS.md/CLAUDE.md、Skills 与扩展注入的真实顺序

分析版本：027a5847901b5dde30270abaa1041046cd2b4b55

核心结论

System prompt 有“资源发现阶段”和“每轮 Agent 启动阶段”两层。ResourceLoader 先发现 SYSTEM.md/APPEND_SYSTEM.md、AGENTS.md/CLAUDE.md、Skills；AgentSession 用 buildSystemPrompt 生成 base system prompt。每次 prompt 前的 before_agent_start 扩展还可以只为当前 turn 覆盖 system prompt。AGENTS.md/CLAUDE.md 是 project_context，不等同于 SYSTEM.md。

1. 先纠正一个术语误区

源码当前自动发现的“上下文文件”候选是 AGENTS.md/AGENTS.MD/CLAUDE.md/CLAUDE.MD；它们从 global agentDir 以及 cwd 到根目录的祖先链加载。SYSTEM.md 是“替换基础 system prompt”的另一条资源通道。两类文件最后都进入 system prompt 字符串，但语义和优先级不同。

2. ResourceLoader 负责发现，不负责最终拼字符串

DefaultResourceLoader 对外暴露 getAgentsFiles、getSkills、getSystemPrompt、getAppendSystemPrompt。loadProjectContextFiles 先取 ~/.pi/agent 下全局 context，再从 cwd 向上遍历祖先目录，并用 unshift 保证父层到子层的顺序。

资源	默认位置/来源	语义
AGENTS.md / CLAUDE.md	~/.pi/agent + cwd 祖先链	项目约束，作为 <project_context> 追加
SYSTEM.md	项目 .pi/SYSTEM.md 或 ~/.pi/agent/SYSTEM.md	替换默认 system prompt 主体
APPEND_SYSTEM.md	项目/全局	追加到主体，不替换
Skills	用户/项目/package/CLI 路径	在 prompt 中列出可读取的技能说明
--system-prompt	CLI/SDK 显式输入	最高显式来源之一，可直接是文本或文件
--append-system-prompt	CLI/SDK	额外 append source

3. buildSystemPrompt 的两条分支

如果 customPrompt 存在，函数不会生成默认 “expert coding assistant” 主体，而是从 customPrompt 开始；但仍会继续 appendSystemPrompt、project context、skills（有 read 工具时）和 current working directory。

如果没有 customPrompt，则构建默认主体：可见工具列表、动态 guidelines、Pi 文档路径，然后追加 append section、project context、skills 和 cwd。也就是说，“SYSTEM.md 替换默认提示词” 并不等于 “屏蔽 AGENTS.md 与 Skills”。

4. 工具集合会反向影响 system prompt

selectedTools 与 toolSnippets 决定 Available tools 列表；guidelines 也会根据工具能力变化。例如有 bash 而没有 grep/find/ls 时，默认 guideline 会提醒用 bash 做文件操作。这说明 system prompt 不是静态常量，而是运行时能力描述。

5. Skills 为什么只有在有 read 时注入

buildSystemPrompt 仅在 read 可用时调用 formatSkillsForPrompt。Skills 通常只把 “名称、描述、路径/使用说明” 放入提示词，模型需要 read 工具再读取完整 SKILL.md。若没有 read，列出不可访问技能反而会制造虚假能力。

6. AgentSession 如何形成 base prompt

AgentSession 在 _buildRuntime/_buildSystemPromptOptions 中收集 cwd、loadedSkills、contextFiles、ResourceLoader.getSystemPrompt、append system prompt、active tools、tool snippets、prompt guidelines，并调用 buildSystemPrompt。结果保存为 _baseSystemPrompt，同时写入 agent.state.systemPrompt。

7. 每一轮还可以被 Extension 改写

AgentSession.prompt 在构造 user AgentMessage 后触发 extensionRunner.emitBeforeAgentStart(expandedText, images, basePrompt, options)。扩展可以返回 messages 和 systemPrompt；如果返回 systemPrompt，则只给当前运行覆盖 agent.state.systemPrompt；否则恢复 _baseSystemPrompt。_runAgentPrompt finally 会清理 override。

```
startup/reload resources
-> ResourceLoader
-> buildSystemPrompt(...)
-> _baseSystemPrompt

per user prompt
-> before_agent_start
  -> optional systemPrompt override
-> Agent run
-> finally clear override
```

8. 项目 Trust 为什么与 Prompt 安全相关

项目本地 .pi settings/extensions/packages 需要 trust 后加载；context files 则在 trust 流程中有独立处理。因为 prompt、extension、tool 都可能影响 Agent 行为，ResourceLoader 明确区分预信任扩展与最终资源集。分析“为什么我的 SYSTEM/Extension 没生效”时必须同时看 trust state。

9. 最终顺序模型

```
Base body:
  custom SYSTEM/--system-prompt ? custom : default Pi prompt
  + APPEND_SYSTEM / --append-system-prompt
  + <project_context> AGENTS/CLAUDE files (global -> ancestors -> cwd)
  + Skills section (if read available)
  + Current working directory

Per-turn:
  before_agent_start may replace effective prompt for this run only
```

10. 常见误判

- 把 AGENTS.md 当作 SYSTEM.md：前者是追加的项目指令，后者替换默认主体。
- 认为 SYSTEM.md 会禁止 Skills/context：源码明确仍然追加。
- 修改 Skill 后没 /reload：ResourceLoader 的资源是有生命周期和缓存的。
- 扩展 before_agent_start 的 system prompt 误以为永久修改：每轮结束会清理 override。
- 工具被禁用但 prompt 仍声称可用：应检查 toolSnippets/active tool 重建流程。

动态验证方案

实验 1：验证资源顺序

假设：全局 context 应先于父目录，再到 cwd

操作：在三个层级创建不同标记的 AGENTS.md，启动并打印 ctx.getSystemPrompt()。

预期：project_context 中顺序从 global/高层到低层 cwd。

若不一致：若不同，检查同目录 CLAUDE/AGENTS 候选优先级与 duplicate path。

实验 2：验证 SYSTEM 替换但 context 保留

假设：SYSTEM.md 只替换主体

操作：创建 .pi/SYSTEM.md 和 AGENTS.md，查看 effective prompt。

预期：默认 Pi 主体消失，但 SYSTEM 内容、project_context 和 cwd 都存在。

若不一致：若项目 SYSTEM 未加载，检查 project trust。

实验 3：验证 per-turn override

假设：before_agent_start 只影响当前 run

操作：扩展第一轮返回 systemPrompt，第二轮不返回。

预期：第一轮使用 override，第二轮恢复 base。

若不一致：若持续存在，检查 `_systemPromptOverride` 清理或扩展每轮都在返回。

源码证据索引

文件	关键符号/用途	本报告结论
packages/coding-agent/src/core/resource-loader.ts	loadProjectContextFiles / getSystemPrompt / reload	资源发现与顺序
packages/coding-agent/src/core/system-prompt.ts	BuildSystemPromptOptions / buildSystemPrompt	最终基础提示词拼装
packages/coding-agent/src/core/agent-session.ts	_buildSystemPromptOptions / prompt / before_agent_start	把资源绑定到 Agent，并做每轮覆盖
packages/coding-agent/docs/usage.md	Context Files / System Prompt Files	仓库行为文档与源码交叉确认
packages/coding-agent/src/core/skills.ts	formatSkillsForPrompt / loadSkills	Skill 提示注入

证据状态：除非单独标注，本报告中的行为结论均来自上述固定 commit 的静态源码与仓库测试/文档交叉验证；未实际启动 Pi、未抓取真实网络请求，因此“运行结果已验证”项为空。

推荐阅读顺序

优先级	文件	阅读目的	精读
1	packages/coding-agent/src/core/resource-loader.ts	先理解资源从哪里来	是
2	packages/coding-agent/src/core/system-prompt.ts	再看拼接顺序	是
3	packages/coding-agent/src/core/agent-session.ts	追 base prompt 如何写入 Agent	是
4	packages/coding-agent/src/core/skills.ts	理解 Skills 是索引而非直接执行	否
5	packages/coding-agent/docs/usage.md	用行为文档核对 SYSTEM/APPEND/Context 语义	否